

```
# due to the replace function, " Hello, Everyone " will be printed
print(str)
str = str.lower()
# the lowercase version " hello, everyone " will be printed
print(str)
str = str.strip()
# without whitespaces, "hello, everyone" will be printed
print(str)
split_str = str.split(",")
# the string has been split around the comma and stripped, so the sec
print(split_str[1].strip())
```

```
    Hello, World
Say Hello
World
    Hello, Everyone
    hello, everyone
hello, everyone
everyone
```

Datatypes

Python **lists** are used to store items in an ordered, indexed variable that can be modified. Like strings, lists have backward and forward indexing. However, instead of accessing characters, we can access whichever object was placed at the index. In order to find an item within the list, the method `index(item)` will return the first index where the item occurs. Or, the `count(item)` method will sum the number of occurrences of the item in the list.

Lists can also be **sliced**, using a similar format to strings. To get the first five elements of a list, we would use `list[:5]`. The last four elements would be `list[-5:]`. The call `list[3:7]` would get the elements from the third position to the sixth.

In order to add an item to a list, we use the `append(item)` function. To put an element in a specific position, the `insert(index, item)` call is used. Conversely, the `remove(item)` function deletes the first occurrence of the element. To empty the entire list, the `clear()` function can be utilized. To reverse the contents of the list, we use the `reverse()` function, and to sort the list we can use the `sort()` function. To join two different lists either the `'+'` operator or the `extend(list2)` function can be used.

Characteristics of the list can also be obtained using functions. The number of total elements in a list is found with the `len(list)` function. The maximum and minimum values are calculated with the `max(list)` and `min(list)` functions, respectively. For lists comprised of only ints or floats, the `sum(list)` function finds the sum of all values. To copy the list we can use `copy()`, and to create a sorted copy of the list we use `sorted(list)`.

To loop through a list, our for loop must use "for item in list" as its header. Then, each item can be modified or used within the loop's body.

```
# the list below is the grades of a class on a math test
grades = [85, 23, 90, 98, 100, 55, 67, 48, 67, 88, 92, 92, 45, 77, 75]
# the call below shows the number of grades, or the number of student
num = len(grades)
print(f"There are {num} students in the class")
# someone joined the class late and had to make up the exam
grades.append(95)
# the teacher gave fifteen points to the person with the lowest grade
lowest = min(grades)
grades.remove(lowest)
grades.append(lowest + 15)
# the teacher wishes to find the five highest scores
grades.sort()
highest = grades[-5:]
print(f"The five highest grades on the test were {highest}")
# the teacher wants to find the average score
```

```

avg = sum(grades)/num
print(f"The average score on the test was {avg}")
# the teacher wishes to award points based on how low a student score
grades.reverse()
bonus = 0
new_grades = []
for grade in grades:
    new_grades.append(grade + bonus)
    bonus += 1
# display the new grades
print(f"Here are the new grades: {new_grades}")
# another class is also reporting their grades
other_grades = [45, 43, 55, 66]
all_grades = new_grades + other_grades
# display the grades
print(f"Here are all the grades: {all_grades}")

```

There are 17 students in the class

The five highest grades on the test were [92, 92, 95, 98, 100]

The average score on the test was 80.05882352941177

Here are the new grades: [100, 99, 97, 95, 96, 95, 94, 92, 85, 84, 85, 85, 79, 80, 69, 63, 61, 55]

Here are all the grades: [100, 99, 97, 95, 96, 95, 94, 92, 85, 84, 85, 85, 79, 80, 69, 63, 61, 55, 45, 43, 55, 66]

Tuples also store multiple, ordered values in a single variable. However, tuples cannot be changed after it has been created, unlike lists. We can check if an item is in the tuple using the 'in' operator, like with lists and strings.

Like lists, the elements of a tuple can be accessed with forward and backward indexing. Additionally, tuples can be sliced to retrieve smaller tuples. The first element of a tuple would be `tuple[0]` and the call `tuple[-5:]` would get the last five elements.



Tuples cannot be changed, so modifying them in any way is difficult. However, we can convert the tuple to a list using the `list(tuple)` function and change the list as necessary. the tuple can be converted back with `tuple(list)` call and assigned back to the original variable. While individual items cannot be added to a tuple, two tuples can be joined together with the '+' operator and assigned to the variable. We can also multiply tuples to increase the instances of each element; for, example, if we perform `tuple * 2`, each element inside the tuple will now be repeated.

Like with lists, the `index(item)` function will find the index of the first occurrence of the element in the tuple and `count(item)` will return the number of instances. Additionally, the `len(tuple)`, `max(tuple)`, and `min(tuple)` will return length, maximum, and minimum values, respectively. With a tuple of all numeric values, the `sum(tuple)` call will return the sum of all values within it. The `sorted(tuple)` function will return a sorted version of the tuple.

Looping over tuples is very similar to looping over lists, our header will be: `for item in tuple`. Through this, we can access but not directly change the tuple's values within the loop's body.

```
# consider this collection of items in a grocery cart
basket = ("apple", "apple", "frozen pizza", "cereal", "milk", "orange")
# here are the associated prices
prices = (1.5, 1.5, 5, 3, 4, 3, 10, 10, 2)
# find the full cost of the cart
total = sum(prices)
print(f"The full cost is {total}")
# the customer must buy mre since there is an extra visitor in her hc
basket *= 2
prices *= 2
# all animal products are now taxed 50%
price_list = list(prices)
price_list.remove(5)
price_list.remove(5)
```

```

price_list.remove(4)
price_list.remove(4)
price_list.remove(10)
price_list.remove(10)
price_list.remove(10)
price_list.remove(10)
price_list.extend([5*1.5, 5*1.5, 4*1.5, 4*1.5, 15, 15, 15, 15])
# the customer is concerned about how many items are above $5
count = 0
for item in price_list:
    if item > 5:
        count += 1
print(f"{count} items are above $5")

```

The full cost is 40.0
8 items are above \$5

Like lists, **sets** can have elements added and removed. Unlike tuples and lists, however, this data type is also unordered and does not allow for duplicate values.

To add an item to a set, we can use `add(item)`. The call `update(set2)` adds a second set's elements to the original set. To remove an item that is definitely in the list the method `remove(item)` can be used. Otherwise, the call `discard(item)` will not raise any errors. `pop()` can be used to remove and obtain a random item from the set. When emptying the entire set, the `clear()` function can be called.

Several operations on sets from set theory are also built into Python such as **union**(other_sets), **intersection**(other_sets), and **difference**(other_sets). Each of these function produces a new set with the given operation performed between the first and second sets. The **symmetric_difference**(set2) method returns a new set which contains all elements in either set but not in intersection. To check whether a set is a **superset**, **subset**, or **disjoint** of

another set, we use the `issuperset(set2)`, `issubset(set2)`, and `isdisjoint(set2)` methods.

Like with the previous data types, the `len(set)` function can be used to find the number of items in the set. `max(set)` and `min(set)` return the maximum and minimum of the set. Furthermore, `sorted(set)` returns a sorted version of the set.

Looping over tuples is also the same as for the other types, and our header will be: `for item in set`. We can perform methods using the item and other set operations in the body.

```
# the students athletes at a highschool
tennis = {"Aiden", "Max", "Carly", "Lily", "Zara", "Ethan", "Frankie"}
soccer = {"Cassie", "Sarah", "Jennie", "Jacob", "Timothy", "River", "Megan"}
track = {"Jacob", "Max", "Lily", "Jackson", "Marie", "Megan", "Ben", "Benjamin"}
# which students play both tennis and run track?
both = tennis.intersection(track)
print(f"These students play both tennis and track: {both}")
# which students play only one sport?
one = tennis.symmetric_difference(track.union(soccer))
print(f"These students play only one sport: {one}")
# how many student athletes are there?
athletes = tennis.union(soccer, track)
print(f"There are {len(athletes)} students athletes")
# three students are friends outside of school. do they all play tennis?
friends = {"Max", "Lily", "Taylor"}
if friends.issubset(tennis):
    print("Yes, they all play tennis")
# do any of the friends play soccer?
if not friends.isdisjoint(soccer):
    print("Yes, they do")
```

These students play both tennis and track: {'Lily', 'Max'}

These students play only one sport: {'Aaron', 'Megan', 'River', 'Marie', 'Aiden', 'Rose', 'Cassie', 'John', 'Jackson', 'Taylor', 'Ethan', 'Ben', 'April', 'Riley', 'Eric', 'Christina', 'Jacob', 'Frankie', 'Sarah', 'Carly', 'Zara', 'Timothy', 'Jennie'}

There are 26 students athletes

Yes, they all play tennis

Yes, they do

Dictionaries, like sets, do not allow for duplicates and can be changed. Unlike sets, however, they are ordered and have indices for access. This data type is comprised of key:value pairs and written in this format as well. To access a value in the dictionary, we can use `dictionary[key]` or the `dictionary.get(key)` method. The `keys()` method will provide a list of all keys if they are not known. For the values, we can call `values()`, and for a tuple list of each pair there is `items()`.

To modify the value associated with a key, either the square brackets can be used like `dictionary[key] = new_value`. Otherwise, the `update({key:value})` call will perform the same change. The same two methods will also add a new key:value pair if the key is not already present within the dictionary. To remove a key:value pair, the `pop(key)` method will both remove the pair and return the associated value. Otherwise, the `popItem()` call removes and returns the last inserted pair. To empty the entire structure, we can use the `clear()` method. The `copy()` method, conversely, will create a copy of the dictionary to set another variable to.

Like with the other datasets, the `len(dictionary)` and `sorted(dictionary)` will give the number of pairs and provide a sorted list of keys, respectively. To create an empty dictionary, a copy, or convert another data structure into a dictionary, we can use `dict(object)`. As always, the `del` keyword will delete any entire dictionary object.

The format for looping over dictionaries is slightly different. We can either loop directly over the dictionary, which is equivalent to looping over its keys or over `dictionary.keys()`. Otherwise, we can loop directly over the values with `values()` or the key:value pairs entirely with `items()`. Depending on the purpose of the loop, different functions can be used in the header.

```
import json
# opening iris json data
with open('iris.json', 'r') as file:
    iris = json.load(file)
# find the average petal length of versicolor irises
sumlength = 0
num = 0
for flower in iris:
    if flower['species'] == 'versicolor':
        sumlength += flower.get('petalLength')
        num += 1
print(f"The average petal length of versicolor irises is {sumlength/r")
# opening webster dictionary data
with open('dictionary.json', 'r') as file:
    dictionary = json.load(file)
# how many words are in this dictionary?
print(f"There are {len(dictionary)} words in this dictionary")
# find the meaning of the longest word
longest = ""
for word in dictionary:
    if len(word) > len(longest):
        longest = word
print(f"The longest word is {longest}")
print(f"Its definition is: {dictionary.get(longest)}")
# remove the longest word from the dictionary
dictionary.pop(longest)
# find and print every word alphabetically before "abacus" in the dic
```



```
keylist = sorted(dictionary)
for key in keylist:
    if key < "abacus":
        print(key)
```

The average petal length of versicolor irises is 4.26

There are 102217 words in this dictionary

The longest word is vickers-maxim automatic machine gun

Its definition is: An automatic machine gun in which the mechanism is worked by the recoil, assisted by the pressure of gases from the muzzle, which expand in a gas chamber against a disk attached to the end of the barrel, thus moving the latter to the rear with increased recoil, and against the front wall of the gas chamber, checking the recoil of the system.

-

--

-able

-ably

-ana

-ance

-ancy

-ant

-arch

-ard

-art

-ate

-ation

-blast

-derm

-dom

-ed

-ee

-en
-ence
-ency
-ent
-er
-escent
-esque
-ess
-est
-et
-ferous
-ful
-fy
-gen
-genous
-gerous
-gram
-graph
-graphy
-grave
-head
-hood
-ible
-ic
-ics
-ide
-in
-ine
-ing
-ion
-ise
-ish
-ism

- ist
- ite
- itis
- ive
- ize
- kin
- less
- let
- ling
- lite
- lith
- logy
- ly
- mancy
- ment
- mere
- meter
- metry
- mo
- morphous
- ness
- ock
- oid
- ol
- oma
- one
- or
- ory
- ose
- our
- ous
- plastic
- plasty

-pod
-poda
-ric
-s
-scope
-ship
-some
-ster
-type
-uret
-ward
-wards
-ways
-yl
a
a b c
a cappella
a cheval
a fortiori
a mensa et thoro
a posteriori
a priori
a-
a-good
a-mornings
a-sea
a-tiptoe
aam
aard-vark
aard-wolf
aaronic
aaronical
ab

ab-
abaca
abacinate
abacination
abaciscus
abacist
aback
abactinal
abaction
abactor
abaculus

Operators

Arithmetic operators are used to perform mathematical operation using numbers. This includes addition ('+'), subtraction ('-'), multiplication ('*'), and division ('/'). Less straightforward operators are the modulo operation ('%'), floor division ('//'), and exponents ('**'). The modulo operation finds the remainder of a division calculation. Conversely, floor division discards the remainder and returns just the integer result.

```
print(5 + 8)
print(6 - 3)
print(2 * 10)
print(8 / 2)
print(13 % 4)
print(8 // 5)
print(2 ** 3)
```

13
3
20
4.0

1
1
8

To compare two different values, we use **comparison** operators. These operators check whether the condition holds and returns a boolean for its truth value. The '=' operator checks if two values are equal, '!=' checks whether they are not equal, '>' checks if the first value is greater, and '<' checks if the second value is greater. For a broader range of functionality, '<=' and '>=' checks if the first value is less than or equal and greater than or equal to the second value, respectively.

```
if 5 < 2:
    print("True")
    print("5 is less than 2")
else:
    print("False")
    print("5 is not less than 2")
print(5<=5)
print(5!=5)
if 0 == 0:
    print("0 is 0")
```

False

5 is not less than 2

True

False

0 is 0

Using **logical** operators, we can combine two conditional statements, like the ones created using comparison operators. The **and** operator returns true if both statements are true and false otherwise, the **or** operator only returns true if at least one statement is true, and the **not** operator returns the opposite boolean value of the conditional statement it is applied to.

```
if 3 >= 2 and 3 <= 4:
    print("True")
if 0 != 0 or 1 == 1:
    print("Or holds")
print(not True)
```

True

Or holds

False

The **membership** operator '**in**' returns true if the given value is found in the object. Conversely, the '**not in**' operator only returns true if the given value cannot be found in the object.

Identity operators check if two objects are the same. The operator '**is**' returns true if the both are the same object and '**is not**' returns true if they are not.

```
fruits = ["apple", "orange", "pear", "banana", "peach"]
print("apple" in fruits)
print("pineapple" not in fruits)
if type(fruits) is list:
    print("fruits is a list")
if fruits is not fruits:
    print("0 is not an int")
```

True

True

fruits is a list

Bitwise operators are used to perform operations in base 2, using bits. The '**&**' operator compares each bit in two given numbers and sets each result bit to 1 if both bits are also 1 at that position. The '**|**' operator sets each result bit to 1 if either of the values' bit is 1 at that location. Similarly, the '**^**' operator sets each result bit to 1 if only one of the values' bit is 1 at that position. The '**~**' operator simply flips all the bits into their opposite value. The leftward shift operator

'<<' shifts all the bits to the left by a given amount, filling in the spaces with zeroes. This is equivalent to multiplying the value by a power of two. Meanwhile, the rightward shift '>>' shifts the bits to the right and divides the value by some power of two.

```
print(20 & 15)
print(5 | 4)
print(10 ^ 2)
print(~16)
print(3 << 2)
print(3 * 2**2)
print(8 >> 1)
print(8 / 2)
```

```
4
5
8
-17
12
12
4
4.0
```

The most basic **assignment** operator is '='. Assignment operators can also combine with arithmetic operators to perform a mathematic operation and assign the value back to a variable. Additionally, bitwise operators can also be combined with the assignment '=' to perform a bitwise operation and assign the new value.

```
count = 1
for i in range(1, 4):
    count *= i
print(count)
count %= 4
```



```
print(count)
count **= 4
print(count)
count //= 3
print(count)
count <=<= 2
print(count)
count ^= 2
print(count)
```

```
6
2
16
5
20
22
```

Functions

Python functions are constructed using the '**def**' keyword. Each function is assigned a name to be called again and, within parentheses, the **parameters** you must pass into it. The header of a function is always: `def function_name(args):`. Within the function body, we can add the code block which uses and changes the parameters as necessary. The body of the function can also contain a **return** value, which will be produced as a final result of the function call. Functions are a way of simplifying code and reducing redundancy, so this should be code that will either be used multiple times or condenses otherwise tedious work.

The methods we have previously used, such as `copy()`, `union(set)`, and `index(list)`, are all built-in functions for python. For user-defined functions, we must use the format specified above. The ability to define our own functions

expands the ability of what our code can do and greatly simplifies longer projects.

```
# this function cubes the parameter
```

```
def cube(num):  
    return num**3
```

```
print(cube(8))  
print(cube(-4))
```

```
#this function says hello to the given name
```

```
def hello (name):  
    print(f"Hello, {name}!")
```

```
hello("World")  
hello("Everyone")  
hello("You")  
hello("Sreeja")
```

```
512
```

```
-64
```

```
Hello, World!
```

```
Hello, Everyone!
```

```
Hello, You!
```

```
Hello, Sreeja!
```